Lineage trace evidence pack

Item	Value
Reference	TRC-20260630-0058
Data point	LCR Liquidity Coverage · ENT-UK lcr_pct
Reported value	150.0 USD m
Reference date	2026-06-30
Result	USD 150m traced to 5 systems of record
Hops	20 (20 proven, 0 code-derived, 0 unresolved)
Generated by	tracer user (tracer)
Generated at	2026-10-16 09:11 UTC

TRC-20260630-0058
tracer user (tracer) · 2026-10-16 09:11 UTC

Hops

Hop	Status	From -> To	Transformation	Evidence	Reconciliation
1	proven	lcr_mart.raw.cashflow -> lcr_mart.stg.cashflow	Calculated as ROUND(c.amount * fx.rate_to_usd, 2); converted using FX rate fx.rate_to_usd.	pipelines/lcr/12_stg_cashflow.sql lines 3–11 · definition sha256 d1efb9ed3026… · captured 2026-10-16 · 16-Oct 08:44:50 · app lcr_shell · user etl_owner · INSERT INTO stg.cashflow (entity_code, flow_type, amount_usd, maturity_date) SELECT UPPER(c.entity_code), c.fl	Recalculated 944,023,680.74 agrees to stored value 944,023,680.74 (tolerance ±0.01) [cashflow.amount_usd]
2	proven	lcr_mart.raw.fx_rate -> lcr_mart.stg.cashflow	Calculated as ROUND(c.amount * fx.rate_to_usd, 2); converted using FX rate fx.rate_to_usd.	pipelines/lcr/12_stg_cashflow.sql lines 3–11 · definition sha256 d1efb9ed3026… · captured 2026-10-16 · 16-Oct 08:44:50 · app lcr_shell · user etl_owner · INSERT INTO stg.cashflow (entity_code, flow_type, amount_usd, maturity_date) SELECT UPPER(c.entity_code), c.fl	Recalculated 944,023,680.74 agrees to stored value 944,023,680.74 (tolerance ±0.01) [cashflow.amount_usd]
3	proven	lcr_mart.raw.deposit_account -> lcr_mart.stg.deposit_account	Calculated as ROUND(d.balance * fx.rate_to_usd, 2); converted using FX rate fx.rate_to_usd.	pipelines/lcr/10_stg_deposit_account.sql lines 3–13 · definition sha256 8a5b752749b7… · captured 2026-10-16 · 16-Oct 08:44:50 · app lcr_shell · user etl_owner · INSERT INTO stg.deposit_account (account_id, entity_code, cpty_type, is_operational, insured, balance_usd) SEL	Recalculated 1,729,380,467.95 agrees to stored value 1,729,380,467.95 (tolerance ±0.01) [deposit_account.balance_usd]
4	proven	lcr_mart.raw.fx_rate -> lcr_mart.stg.deposit_account	Calculated as ROUND(d.balance * fx.rate_to_usd, 2); converted using FX rate fx.rate_to_usd.	pipelines/lcr/10_stg_deposit_account.sql lines 3–13 · definition sha256 8a5b752749b7… · captured 2026-10-16 · 16-Oct 08:44:50 · app lcr_shell · user etl_owner · INSERT INTO stg.deposit_account (account_id, entity_code, cpty_type, is_operational, insured, balance_usd) SEL	Recalculated 1,729,380,467.95 agrees to stored value 1,729,380,467.95 (tolerance ±0.01) [deposit_account.balance_usd]
5	proven	lcr_mart.raw.lcr_param -> lcr_mart.stg.lcr_param	Calculated as CAST(p.factor AS DECIMAL).	pipelines/lcr/13_stg_lcr_param.sql lines 3–6 · definition sha256 f22c0f6c18ca… · captured 2026-10-16 · 16-Oct 08:44:50 · app lcr_shell · user etl_owner · INSERT INTO stg.lcr_param (item_code, factor) SELECT p.item_code, p.factor::numeric FROM raw.lcr_param p;	Recalculated 4.95 agrees to stored value 4.95 (tolerance ±0.01) [lcr_param.factor]
6	proven	lcr_mart.raw.fx_rate -> lcr_mart.stg.security_holding	Calculated as ROUND(s.market_value * fx.rate_to_usd, 2); converted using FX rate fx.rate_to_usd.	pipelines/lcr/11_stg_security_holding.sql lines 3–12 · definition sha256 584db098c4db… · captured 2026-10-16 · 16-Oct 08:44:50 · app lcr_shell · user etl_owner · INSERT INTO stg.security_holding (isin, entity_code, asset_class, is_encumbered, mv_usd) SELECT s.isin, UPPER(	Recalculated 754,173,898.36 agrees to stored value 754,173,898.36 (tolerance ±0.01) [security_holding.mv_usd]
7	proven	lcr_mart.raw.security_holding -> lcr_mart.stg.security_holding	Calculated as ROUND(s.market_value * fx.rate_to_usd, 2); converted using FX rate fx.rate_to_usd.	pipelines/lcr/11_stg_security_holding.sql lines 3–12 · definition sha256 584db098c4db… · captured 2026-10-16 · 16-Oct 08:44:50 · app lcr_shell · user etl_owner · INSERT INTO stg.security_holding (isin, entity_code, asset_class, is_encumbered, mv_usd) SELECT s.isin, UPPER(	Recalculated 754,173,898.36 agrees to stored value 754,173,898.36 (tolerance ±0.01) [security_holding.mv_usd]
8	proven	lcr_mart.stg.lcr_param -> lcr_mart.core.fact_hqla	Sums s.mv_usd grouped by s.entity_code, hqla_level, p.factor; filtered to NOT s.is_encumbered; filtered to s.asset_class IN ('L1_SOVEREIGN', 'L1_CENTRAL_BANK', 'L2A_GOV', 'L2A_COVERED', 'L2B_RMBS', 'L2B_CORP_BOND').	pipelines/lcr/20_core_fact_hqla.sql lines 3–22 · definition sha256 90941ef521db… · captured 2026-10-16 · 16-Oct 08:44:50 · app lcr_shell · user etl_owner · INSERT INTO core.fact_hqla (entity, hqla_level, mv_usd, haircut, weighted_usd) SELECT s.entity_code, CASE WHEN	Recalculated 0.65 agrees to stored value 0.65 (tolerance ±0.01) [fact_hqla.haircut]

Hop	Status	From -> To	Transformation	Evidence	Reconciliation
9	proven	lcr_mart.stg.security_holding -> lcr_mart.core.fact_hqla	Sums s.mv_usd grouped by s.entity_code, hqla_level, p.factor; filtered to NOT s.is_encumbered; filtered to s.asset_class IN ('L1_SOVEREIGN', 'L1_CENTRAL_BANK', 'L2A_GOV', 'L2A_COVERED', 'L2B_RMBS', 'L2B_CORP_BOND').	pipelines/lcr/20_core_fact_hqla.sql lines 3–22 · definition sha256 90941ef521db… · captured 2026-10-16 · 16-Oct 08:44:50 · app lcr_shell · user etl_owner · INSERT INTO core.fact_hqla (entity, hqla_level, mv_usd, haircut, weighted_usd) SELECT s.entity_code, CASE WHEN	Recalculated 530,595,633.08 agrees to stored value 530,595,633.08 (tolerance ±0.01) [fact_hqla.mv_usd]
10	proven	lcr_mart.stg.cashflow -> lcr_mart.core.fact_inflow	Sums c.amount_usd grouped by c.entity_code, c.flow_type, p.factor; filtered to c.flow_type LIKE 'IN%'; filtered to c.maturity_date > CAST('2026-06-30' AS DATE); filtered to c.maturity_date <= CAST('2026-07-30' AS DATE).	pipelines/lcr/22_core_fact_inflow.sql lines 3–15 · definition sha256 2068c350d698… · captured 2026-10-16 · 16-Oct 08:44:50 · app lcr_shell · user etl_owner · INSERT INTO core.fact_inflow (entity, category, balance_usd, inflow_rate, inflow_usd) SELECT c.entity_code, c.	Recalculated 546,509,177.03 agrees to stored value 546,509,177.03 (tolerance ±0.01) [fact_inflow.balance_usd]
11	proven	lcr_mart.stg.lcr_param -> lcr_mart.core.fact_inflow	Sums c.amount_usd grouped by c.entity_code, c.flow_type, p.factor; filtered to c.flow_type LIKE 'IN%'; filtered to c.maturity_date > CAST('2026-06-30' AS DATE); filtered to c.maturity_date <= CAST('2026-07-30' AS DATE).	pipelines/lcr/22_core_fact_inflow.sql lines 3–15 · definition sha256 2068c350d698… · captured 2026-10-16 · 16-Oct 08:44:50 · app lcr_shell · user etl_owner · INSERT INTO core.fact_inflow (entity, category, balance_usd, inflow_rate, inflow_usd) SELECT c.entity_code, c.	Recalculated 4.5 agrees to stored value 4.5 (tolerance ±0.01) [fact_inflow.inflow_rate]
12	proven	lcr_mart.stg.cashflow -> lcr_mart.core.fact_outflow	Sums c.amount_usd grouped by c.entity_code, c.flow_type, p.factor; filtered to c.flow_type LIKE 'OUT%'; filtered to c.maturity_date > CAST('2026-06-30' AS DATE); filtered to c.maturity_date <= CAST('2026-07-30' AS DATE).	pipelines/lcr/21_core_fact_outflow.sql lines 26–38 · definition sha256 856c0288e8eb… · captured 2026-10-16 · 16-Oct 08:44:50 · app lcr_shell · user etl_owner · INSERT INTO core.fact_outflow (entity, category, balance_usd, runoff_rate, outflow_usd) SELECT c.entity_code,	Recalculated 1,951,320,075.32 agrees to stored value 1,951,320,075.32 (tolerance ±0.01) [fact_outflow.balance_usd]
13	proven	lcr_mart.stg.deposit_account -> lcr_mart.core.fact_outflow	Sums d.balance_usd grouped by d.entity_code, category, p.factor.	pipelines/lcr/21_core_fact_outflow.sql lines 3–24 · definition sha256 6f6a50f876d5… · captured 2026-10-16 · 16-Oct 08:44:50 · app lcr_shell · user etl_owner · INSERT INTO core.fact_outflow (entity, category, balance_usd, runoff_rate, outflow_usd) SELECT d.entity_code,	Recalculated 1,951,320,075.32 agrees to stored value 1,951,320,075.32 (tolerance ±0.01) [fact_outflow.balance_usd]
14	proven	lcr_mart.stg.lcr_param -> lcr_mart.core.fact_outflow	Sums c.amount_usd grouped by c.entity_code, c.flow_type, p.factor; filtered to c.flow_type LIKE 'OUT%'; filtered to c.maturity_date > CAST('2026-06-30' AS DATE); filtered to c.maturity_date <= CAST('2026-07-30' AS DATE).	pipelines/lcr/21_core_fact_outflow.sql lines 26–38 · definition sha256 856c0288e8eb… · captured 2026-10-16 · 16-Oct 08:44:50 · app lcr_shell · user etl_owner · INSERT INTO core.fact_outflow (entity, category, balance_usd, runoff_rate, outflow_usd) SELECT c.entity_code,	Recalculated 8.4 agrees to stored value 8.4 (tolerance ±0.01) [fact_outflow.runoff_rate]
15	proven	lcr_mart.stg.lcr_param -> lcr_mart.core.fact_outflow	Sums d.balance_usd grouped by d.entity_code, category, p.factor.	pipelines/lcr/21_core_fact_outflow.sql lines 3–24 · definition sha256 6f6a50f876d5… · captured 2026-10-16 · 16-Oct 08:44:50 · app lcr_shell · user etl_owner · INSERT INTO core.fact_outflow (entity, category, balance_usd, runoff_rate, outflow_usd) SELECT d.entity_code,	Recalculated 8.4 agrees to stored value 8.4 (tolerance ±0.01) [fact_outflow.runoff_rate]

Hop	Status	From -> To	Transformation	Evidence	Reconciliation
16	proven	lcr_mart.core.fact_hqla -> lcr_mart.rpt.lcr_summary	Calculated as ROUND(h.hqla / (o.outflows - LEAST(i.inflows, 0.75 * o.outflows)) * 100, 2).	pipelines/lcr/30_rpt_lcr_summary.sql lines 3–14 · definition sha256 1ae123e35b65… · captured 2026-10-16 · 16-Oct 08:44:50 · app lcr_shell · user etl_owner · INSERT INTO rpt.lcr_summary (entity, hqla_usd, outflows_usd, inflows_usd, net_outflows_usd, lcr_pct) SELECT h.	Recalculated 414.6 agrees to stored value 414.6 (tolerance ±0.01) [lcr_summary.hqla_usd]
17	proven	lcr_mart.core.fact_inflow -> lcr_mart.rpt.lcr_summary	Calculated as ROUND(h.hqla / (o.outflows - LEAST(i.inflows, 0.75 * o.outflows)) * 100, 2).	pipelines/lcr/30_rpt_lcr_summary.sql lines 3–14 · definition sha256 1ae123e35b65… · captured 2026-10-16 · 16-Oct 08:44:50 · app lcr_shell · user etl_owner · INSERT INTO rpt.lcr_summary (entity, hqla_usd, outflows_usd, inflows_usd, net_outflows_usd, lcr_pct) SELECT h.	Recalculated 129.11 agrees to stored value 129.11 (tolerance ±0.01) [lcr_summary.inflows_usd]
18	proven	lcr_mart.core.fact_outflow -> lcr_mart.rpt.lcr_summary	Calculated as ROUND(h.hqla / (o.outflows - LEAST(i.inflows, 0.75 * o.outflows)) * 100, 2).	pipelines/lcr/30_rpt_lcr_summary.sql lines 3–14 · definition sha256 1ae123e35b65… · captured 2026-10-16 · 16-Oct 08:44:50 · app lcr_shell · user etl_owner · INSERT INTO rpt.lcr_summary (entity, hqla_usd, outflows_usd, inflows_usd, net_outflows_usd, lcr_pct) SELECT h.	Recalculated 405.51 agrees to stored value 405.51 (tolerance ±0.01) [lcr_summary.outflows_usd]
19	proven	lcr_mart.rpt.lcr_summary -> lcr_mart.rpt.v_lcr_summary	Copied from lcr_summary.lcr_pct; assembled from 2 UNION branches.	lcr_mart: view rpt.v_lcr_summary · definition sha256 f9fec64155d9… · captured 2026-10-16	Recalculated 150 agrees to stored value 150 (tolerance ±0.01) [v_lcr_summary.lcr_pct]
20	proven	lcr_mart.rpt.v_lcr_summary -> superset.LCR Liquidity Coverage	Superset dataset 'lcr_liquidity_summary' is a virtual dataset over SQL; the chart displays it as a table.	Superset chart 24 'LCR Liquidity Coverage' -> dataset lcr_liquidity_summary	Superset chart data re-run returned 150.0 for the selected keys; rpt.v_lcr_summary.lcr_pct sums to 150.0

SQL excerpts

Hop 1: INSERT INTO stg.cashflow (entity_code, flow_type, amount_usd, maturity_date) SELECT UPPER(c.entity_code), c.flow_type, ROUND(c.amount * fx.rate_to_usd, 2), c.maturity_date FROM raw.cashflow c JOIN raw.fx_rate fx ON fx.ccy = c.ccy AND fx.rate_date = DATE '2026-06-30';

Hop 2: INSERT INTO stg.cashflow (entity_code, flow_type, amount_usd, maturity_date) SELECT UPPER(c.entity_code), c.flow_type, ROUND(c.amount * fx.rate_to_usd, 2), c.maturity_date FROM raw.cashflow c JOIN raw.fx_rate fx ON fx.ccy = c.ccy AND fx.rate_date = DATE '2026-06-30';

Hop 3: INSERT INTO stg.deposit_account (account_id, entity_code, cpty_type, is_operational, insured, balance_usd) SELECT d.account_id, UPPER(d.entity_code), UPPER(d.cpty_type), d.is_operational = 'Y', d.insured = 'Y', ROUND(d.balance * fx.rate_to_usd, 2) FROM raw.deposit_account d JOIN raw.fx_rate fx ON fx.ccy = d.ccy AND fx.rate_date = DATE '2026-06-30';

Hop 4: INSERT INTO stg.deposit_account (account_id, entity_code, cpty_type, is_operational, insured, balance_usd) SELECT d.account_id, UPPER(d.entity_code), UPPER(d.cpty_type), d.is_operational = 'Y', d.insured = 'Y', ROUND(d.balance * fx.rate_to_usd, 2) FROM raw.deposit_account d JOIN raw.fx_rate fx ON fx.ccy = d.ccy AND fx.rate_date = DATE '2026-06-30';

Hop 5: INSERT INTO stg.lcr_param (item_code, factor) SELECT p.item_code, p.factor::numeric FROM raw.lcr_param p;

Hop 6: INSERT INTO stg.security_holding (isin, entity_code, asset_class, is_encumbered, mv_usd) SELECT s.isin, UPPER(s.entity_code), s.asset_class, s.is_encumbered = 'Y', ROUND(s.market_value * fx.rate_to_usd, 2) FROM raw.security_holding s JOIN raw.fx_rate fx ON fx.ccy = s.ccy AND fx.rate_date = DATE '2026-06-30';

Hop 7: INSERT INTO stg.security_holding (isin, entity_code, asset_class, is_encumbered, mv_usd) SELECT s.isin, UPPER(s.entity_code), s.asset_class, s.is_encumbered = 'Y', ROUND(s.market_value * fx.rate_to_usd, 2) FROM raw.security_holding s JOIN raw.fx_rate fx ON fx.ccy = s.ccy AND fx.rate_date = DATE '2026-06-30';

Hop 8: INSERT INTO core.fact_hqla (entity, hqla_level, mv_usd, haircut, weighted_usd) SELECT s.entity_code, CASE WHEN s.asset_class IN ('L1_SOVEREIGN', 'L1_CENTRAL_BANK') THEN '1' WHEN s.asset_class IN ('L2A_GOV', 'L2A_COVERED') THEN '2A' ELSE '2B' END AS hqla_level, SUM(s.mv_usd) AS mv_usd, p.factor AS haircut, SUM(s.mv_usd) * (1 - p.factor) AS weighted_usd FROM stg.security_holding s JOIN stg.lcr_param p ON p.item_code = 'HQLA_L' || CASE WHEN s.asset_class IN ('L1_SOVEREIGN', 'L1_CENTRAL_BANK') THEN '1' WHEN s.asset_class IN ('L2A_GOV', 'L2A_COVERED') THEN '2A' ELSE '2B' END WHERE NOT s.is_encumbered AND s.asset_class IN ('L1_SOVEREIGN', 'L1_CENTRAL_BANK', 'L2A_GOV', 'L2A_COVERED', 'L2B_RMBS', 'L2B_CORP_BOND') GROUP BY s.entity_code, 2, p.factor;

TRC-20260630-0058 · tracer user (tracer) · 2026-10-16 09:11 UTC

Hop 9: INSERT INTO core.fact_hqla (entity, hqla_level, mv_usd, haircut, weighted_usd) SELECT s.entity_code, CASE WHEN s.asset_class IN ('L1_SOVEREIGN', 'L1_CENTRAL_BANK') THEN '1' WHEN s.asset_class IN ('L2A_GOV', 'L2A_COVERED') THEN '2A' ELSE '2B' END AS hqla_level, SUM(s.mv_usd) AS mv_usd, p.factor AS haircut, SUM(s.mv_usd) * (1 - p.factor) AS weighted_usd FROM stg.security_holding s JOIN stg.lcr_param p ON p.item_code = 'HQLA_L' || CASE WHEN s.asset_class IN ('L1_SOVEREIGN', 'L1_CENTRAL_BANK') THEN '1' WHEN s.asset_class IN ('L2A_GOV', 'L2A_COVERED') THEN '2A' ELSE '2B' END WHERE NOT s.is_encumbered AND s.asset_class IN ('L1_SOVEREIGN', 'L1_CENTRAL_BANK', 'L2A_GOV', 'L2A_COVERED', 'L2B_RMBS', 'L2B_CORP_BOND') GROUP BY s.entity_code, 2, p.factor;

Hop 10: INSERT INTO core.fact_inflow (entity, category, balance_usd, inflow_rate, inflow_usd) SELECT c.entity_code, c.flow_type AS category, SUM(c.amount_usd) AS balance_usd, p.factor AS inflow_rate, SUM(c.amount_usd) * p.factor AS inflow_usd FROM stg.cashflow c JOIN stg.lcr_param p ON p.item_code = c.flow_type WHERE c.flow_type LIKE 'IN%' AND c.maturity_date > DATE '2026-06-30' AND c.maturity_date <= DATE '2026-07-30' GROUP BY c.entity_code, c.flow_type, p.factor;

Hop 11: INSERT INTO core.fact_inflow (entity, category, balance_usd, inflow_rate, inflow_usd) SELECT c.entity_code, c.flow_type AS category, SUM(c.amount_usd) AS balance_usd, p.factor AS inflow_rate, SUM(c.amount_usd) * p.factor AS inflow_usd FROM stg.cashflow c JOIN stg.lcr_param p ON p.item_code = c.flow_type WHERE c.flow_type LIKE 'IN%' AND c.maturity_date > DATE '2026-06-30' AND c.maturity_date <= DATE '2026-07-30' GROUP BY c.entity_code, c.flow_type, p.factor;

Hop 12: INSERT INTO core.fact_outflow (entity, category, balance_usd, runoff_rate, outflow_usd) SELECT c.entity_code, c.flow_type AS category, SUM(c.amount_usd) AS balance_usd, p.factor AS runoff_rate, SUM(c.amount_usd) * p.factor AS outflow_usd FROM stg.cashflow c JOIN stg.lcr_param p ON p.item_code = c.flow_type WHERE c.flow_type LIKE 'OUT%' AND c.maturity_date > DATE '2026-06-30' AND c.maturity_date <= DATE '2026-07-30' GROUP BY c.entity_code, c.flow_type, p.factor;

Hop 13: INSERT INTO core.fact_outflow (entity, category, balance_usd, runoff_rate, outflow_usd) SELECT d.entity_code, CASE WHEN d.cpty_type = 'RETAIL' AND d.insured THEN 'RET_STABLE' WHEN d.cpty_type = 'RETAIL' THEN 'RET_LESS_STABLE' WHEN d.is_operational THEN 'WHOLESALE_OPERATIONAL' WHEN d.cpty_type = 'FIN' THEN 'FIN_NON_OPERATIONAL' ELSE 'CORP_NON_OPERATIONAL' END AS category, SUM(d.balance_usd) AS balance_usd, p.factor AS runoff_rate, SUM(d.balance_usd) * p.factor AS outflow_usd FROM stg.deposit_account d JOIN stg.lcr_param p ON p.item_code = CASE WHEN d.cpty_type = 'RETAIL' AND d.insured THEN 'RET_STABLE' WHEN d.cpty_type = 'RETAIL' THEN 'RET_LESS_STABLE' WHEN d.is_operational THEN 'WHOLESALE_OPERATIONAL' WHEN d.cpty_type = 'FIN' THEN 'FIN_NON_OPERATIONAL' ELSE 'CORP_NON_OPERATIONAL' END GROUP BY d.entity_code, 2, p.factor;

Hop 14: INSERT INTO core.fact_outflow (entity, category, balance_usd, runoff_rate, outflow_usd) SELECT c.entity_code, c.flow_type AS category, SUM(c.amount_usd) AS balance_usd, p.factor AS runoff_rate, SUM(c.amount_usd) * p.factor AS outflow_usd FROM stg.cashflow c JOIN stg.lcr_param p ON p.item_code = c.flow_type WHERE c.flow_type LIKE 'OUT%' AND c.maturity_date > DATE '2026-06-30' AND c.maturity_date <= DATE '2026-07-30' GROUP BY c.entity_code, c.flow_type, p.factor;

Hop 15: INSERT INTO core.fact_outflow (entity, category, balance_usd, runoff_rate, outflow_usd) SELECT d.entity_code, CASE WHEN d.cpty_type = 'RETAIL' AND d.insured THEN 'RET_STABLE' WHEN d.cpty_type = 'RETAIL' THEN 'RET_LESS_STABLE' WHEN d.is_operational THEN 'WHOLESALE_OPERATIONAL' WHEN d.cpty_type = 'FIN' THEN 'FIN_NON_OPERATIONAL' ELSE 'CORP_NON_OPERATIONAL' END AS category, SUM(d.balance_usd) AS balance_usd, p.factor AS runoff_rate, SUM(d.balance_usd) * p.factor AS outflow_usd FROM stg.deposit_account d JOIN stg.lcr_param p ON p.item_code = CASE WHEN d.cpty_type = 'RETAIL' AND d.insured THEN 'RET_STABLE' WHEN d.cpty_type = 'RETAIL' THEN 'RET_LESS_STABLE' WHEN d.is_operational THEN 'WHOLESALE_OPERATIONAL' WHEN d.cpty_type = 'FIN' THEN 'FIN_NON_OPERATIONAL' ELSE 'CORP_NON_OPERATIONAL' END GROUP BY d.entity_code, 2, p.factor;

Hop 16: INSERT INTO rpt.lcr_summary (entity, hqla_usd, outflows_usd, inflows_usd, net_outflows_usd, lcr_pct) SELECT h.entity, ROUND(h.hqla / 1000000, 2) AS hqla_usd, ROUND(o.outflows / 1000000, 2) AS outflows_usd, ROUND(i.inflows / 1000000, 2) AS inflows_usd, ROUND((o.outflows - LEAST(i.inflows, 0.75 * o.outflows)) / 1000000, 2) AS net_outflows_usd, ROUND(h.hqla / (o.outflows - LEAST(i.inflows, 0.75 * o.outflows)) * 100, 2) AS lcr_pct FROM (SELECT entity, SUM(weighted_usd) AS hqla FROM core.fact_hqla GROUP BY entity) h JOIN (SELECT entity, SUM(outflow_usd) AS outflows FROM core.fact_outflow GROUP BY entity) o ON o.entity = h.entity JOIN (SELECT entity, SUM(inflow_usd) AS inflows FROM core.fact_inflow GROUP BY entity) i ON i.entity = h.entity;

Hop 17: INSERT INTO rpt.lcr_summary (entity, hqla_usd, outflows_usd, inflows_usd, net_outflows_usd, lcr_pct) SELECT h.entity, ROUND(h.hqla / 1000000, 2) AS hqla_usd, ROUND(o.outflows / 1000000, 2) AS outflows_usd, ROUND(i.inflows / 1000000, 2) AS inflows_usd, ROUND((o.outflows - LEAST(i.inflows, 0.75 * o.outflows)) / 1000000, 2) AS net_outflows_usd, ROUND(h.hqla / (o.outflows - LEAST(i.inflows, 0.75 * o.outflows)) * 100, 2) AS lcr_pct FROM (SELECT entity, SUM(weighted_usd) AS hqla FROM core.fact_hqla GROUP BY entity) h JOIN (SELECT entity, SUM(outflow_usd) AS outflows FROM core.fact_outflow GROUP BY entity) o ON o.entity = h.entity JOIN (SELECT entity, SUM(inflow_usd) AS inflows FROM core.fact_inflow GROUP BY entity) i ON i.entity = h.entity;

Hop 18: INSERT INTO rpt.lcr_summary (entity, hqla_usd, outflows_usd, inflows_usd, net_outflows_usd, lcr_pct) SELECT h.entity, ROUND(h.hqla / 1000000, 2) AS hqla_usd, ROUND(o.outflows / 1000000, 2) AS outflows_usd, ROUND(i.inflows / 1000000, 2) AS inflows_usd, ROUND((o.outflows - LEAST(i.inflows, 0.75 * o.outflows)) / 1000000, 2) AS net_outflows_usd, ROUND(h.hqla / (o.outflows - LEAST(i.inflows, 0.75 * o.outflows)) * 100, 2) AS lcr_pct FROM (SELECT entity, SUM(weighted_usd) AS hqla FROM core.fact_hqla GROUP BY entity) h JOIN (SELECT entity, SUM(outflow_usd) AS outflows FROM core.fact_outflow GROUP BY entity) o ON o.entity = h.entity JOIN (SELECT entity, SUM(inflow_usd) AS inflows FROM core.fact_inflow GROUP BY entity) i ON

i.entity = h.entity;

Hop 19: CREATE VIEW "rpt"."v_lcr_summary" AS SELECT lcr_summary.entity, lcr_summary.hqla_usd, lcr_summary.outflows_usd, lcr_summary.inflows_usd, lcr_summary.net_outflows_usd, lcr_summary.lcr_pct FROM rpt.lcr_summary UNION ALL SELECT 'GROUP'::text AS entity, sum(lcr_summary.hqla_usd) AS hqla_usd, sum(lcr_summary.outflows_usd) AS outflows_usd, sum(lcr_summary.inflows_usd) AS inflows_usd, sum(lcr_summary.net_outflows_usd) AS net_outflows_usd, round(((sum(lcr_summary.hqla_usd) / sum(lcr_summary.net_outflows_usd)) * (100)::numeric), 2) AS lcr_pct FROM rpt.lcr_summary

Hop 20: SELECT entity AS entity, hqla_usd AS hqla_usd, outflows_usd AS outflows_usd, inflows_usd AS inflows_usd, net_outflows_usd AS net_outflows_usd, lcr_pct AS lcr_pct FROM rpt.v_lcr_summary ORDER BY (entity = 'GROUP'), entity LIMIT 1000

TRC-20260630-0058
tracer user (tracer) · 2026-10-16 09:11 UTC